ICS 2026 测验 1 回课

周泽坤

2026.10

2.1 位运算：题面与思路

题面

```
unsigned char x = 0xA6;
unsigned char a = x & 0x0F;
unsigned char b = x ^ 0x80;
```

求 a, b 的值，均用十六进制表示。

我的解题思路

十六进制每一位恰好对应 4 个二进制位。先写成二进制，再逐位执行与、异或，最不容易出错。

读题抓手

1. 0x0F 只覆盖低 4 位；
2. 0x80 只覆盖最高位；
3. 分别回忆“与”和“异或”遇到 1 时的作用。

2.1 位运算：推导与知识点

逐位推导

$$
\begin{array}{rcl}
x & = & 1010\,0110 \\
\& \ \texttt{0x0F} & = & 0000\,1111 \\
\hline
a & = & 0000\,0110 = \texttt{0x06}
\end{array}
$$

$$
\begin{array}{rcl}
x & = & 1010\,0110 \\
\oplus \ \texttt{0x80} & = & 1000\,0000 \\
\hline
b & = & 0010\,0110 = \texttt{0x26}
\end{array}
$$

答案

$a = \texttt{0x06}$, $b = \texttt{0x26}$.

对应知识点

- 与 1 保留、与 0 清零；
- 异或 1 翻转、异或 0 保持。

易错点

异或 1 是“翻转”，不是恒定置 1。

2.2 移位

题面

对 8 位二进制数

$$10110100$$

分别右移 2 位，移出的位丢弃：

1. 逻辑右移的结果；
2. 算术右移的结果。

均用 8 位二进制表示。

我的解题思路

两种右移都会删掉末尾两位 00，区别只在左侧补什么：逻辑右移补 0，算术右移复制原符号位 1。

逻辑：$10110100 \longrightarrow \mathtt{00101101}$,

算术：$10110100 \longrightarrow \mathtt{11101101}$.

对应知识点与易错点

逻辑右移适合把位串看作无符号数；算术右移保持有符号数的符号。
不要把“算术右移”误写成左侧始终补 1，它补的是原最高位。

2.3 整数：题面与思路

题面

假设 int 和 unsigned int 均为 32 位：

```
unsigned int count = 3U;
int change = -5;
int a = (change < count);
unsigned int next = count + change;
int b = (next > count);
```

求十进制表示的 a 与 b。

我的解题思路

先找出混合类型表达式。两边分别是 int 与 unsigned int，比较时优先级为将有符号数转化为无符号数，所以先把 -5 转成 32 位无符号数，再比较、相加。

解题顺序

1. 确定通常算术转换后的类型；
2. 写出 -5 的无符号值；
3. 分别计算比较与模 2^{32} 加法。

2.3 整数：推导与知识点

推导

在 32 位无符号表示中，

$$-5 \longrightarrow 2^{32}-5 = 4294967291 = \texttt{0xFFFFFFFB}.$$

因而

$$\begin{aligned} a &= (4294967291 < 3) = 0, \\ next &= (3 + 4294967291) \bmod 2^{32} \\ &= 4294967294 = \texttt{0xFFFFFFFE}, \\ b &= (4294967294 > 3) = 1. \end{aligned}$$

答案

$$a = 0, \qquad b = 1.$$

对应知识点

同等级的 `int` 与 `unsigned int` 混合时，前者转成后者；无符号运算按模 2^{32} 进行。

易错点

-5 不再以负数身份参与比较，而会变成很大的无符号数。

3.1 IEEE 754 标准：题面与思路

题面

IEEE 754 单精度数的编码为

$$\texttt{0xC0E00000}.$$

按照

$$v = (-1)^s \times M \times 2^E,$$

求：

1. 实际指数 E；
2. 该数的十进制值。

我的解题思路

先把 32 位编码依次切成 $s/exp/frac$，再分别计算符号、实际指数和尾数，最后代入数值公式。

字段切分

$$\texttt{0xC0E00000} = \underbrace{1}_{s}\ \underbrace{10000001}_{exp}\ \underbrace{1100\ldots0}_{frac}.$$

解题顺序

$s \to e \to E = e-127 \to M \to (-1)^s M 2^E$。

3.1 IEEE 754 标准：推导与知识点

推导

$$
\begin{aligned}
e &= (10000001)_2 = 129, \\
E &= 129 - 127 = 2, \\
M &= (1.11)_2 = 1.75, \\
v &= (-1)^1(1.75)2^2 = -7.
\end{aligned}
$$

答案

$$E = 2, \qquad v = -7.$$

对应知识点

规格化单精度数的偏置量为 127；尾数含有不写入编码的隐藏首位 1。

易错点

阶码字段跨越十六进制位边界，不能把整个高字节直接当阶码。

3.2 IEEE 754 特殊值

题面

将以下两个 32 位编码按 IEEE 754 单精度格式解释：

$$\texttt{0xFF800000} \longrightarrow ?$$

$$\texttt{0x7FC00000} \longrightarrow ?$$

我的解题思路

看到阶码全为 1，立即进入“特殊值分类”，不再按普通 $M \times 2^E$ 计算；接着只检查小数域是否为 0。

编码	$exp/frac$	结果
0xFF800000	全 1 / 全 0	$-\infty$
0x7FC00000	全 1 / 非 0	**NaN**

对应知识点与易错点

exp 全 1 且 $frac = 0$ 表示正负无穷；exp 全 1 且 $frac \neq 0$ 表示 NaN。NaN 不是“一个很大的有限数”。

3.3 舍入规则

题面

按 IEEE 754 默认的“向偶数舍入”，将下列二进制数保留小数点后两位：

(1) 10.10100

(2) 11.01011

我的解题思路

在保留位后画线：先判断被舍弃部分小于、等于还是大于半个 ULP；只有恰好一半时才检查“偶”。

$$10.10|100 \xrightarrow[\text{保留末位为 0}]{\text{恰好一半}} 10.10,$$

$$11.01|011 \xrightarrow{\text{小于一半}} 11.01.$$

对应知识点与易错点

“偶数”指保留结果的最低有效位为 **0**，不是看十进制整数部分。非中间情形仍按就近舍入。

3.4 浮点数格式：题面与思路

题面

某 8 位浮点格式为

$$\underbrace{s}_{1}\ \underbrace{exp}_{4}\ \underbrace{frac}_{3}, \qquad Bias = 7,$$

并采用向偶数舍入。给定

$$a = (1.100)_2 2^1, \qquad b = (1.110)_2 2^1,$$

求 $a \times b$ 的 8 位十六进制编码。

我的解题思路

按“精确相乘 $\rightarrow$ 规格化 $\rightarrow$ 舍入 $\rightarrow$ 加偏置编码”的顺序，避免过早舍入。

解题检查表

1. 先保留精确乘积；
2. 尾数规格化到 $[1, 2)$；
3. 小数域保留 3 位并向偶数舍入；
4. 用 $exp = E + Bias$ 写出最终位串。

3.4 浮点数格式：推导与知识点

推导与编码

$$a = 3, \quad b = 3.5,$$

$$ab = 10.5 = (1.0101)_2 2^3$$

$$\xrightarrow{\text{保留 3 位、向偶}} (1.010)_2 2^3.$$

$$exp = 3 + 7 = (1010)_2, \qquad frac = 010.$$

$$0|1010|010 = 01010010 = \boxed{\texttt{0x52}}.$$

答案

最终编码为

$$\texttt{0x52}.$$

对应知识点

阶码字段保存 $E + Bias$；小数域只保存隐藏首位 1 后面的部分。

易错点

被舍弃部分恰为一半，保留末位为 0，所以不进位。

3.5 浮点运算：题面与思路

题面

每次运算均按 IEEE 754 单精度向偶数舍入：

```
float x = 16777216.0f; // 2^24
float y = 1.0f;
float a = x + y;
float b = (x + y) - x;
float c = (x - x) + y;
```

求 b 和 c。

我的解题思路

先问：x 附近两个相邻 `float` 相差多少？在 2^{24} 处向上的间距是 2，故 $x+1$ 正好位于两个可表示数中间。

关键观察

单精度有效精度为 24 位：

$$2^{24}, \quad \underbrace{2^{24}+1}_{\text{正中间}}, \quad 2^{24}+2.$$

因此应先确定 $x+y$ 的舍入结果，再继续后续减法。

3.5 浮点运算：推导与知识点

推导

$$x + y = 2^{24} + 1 \xrightarrow{\text{向偶数舍入}} 2^{24} = x.$$

$$b = (x + y) - x = x - x = 0.0,$$
$$c = (x - x) + y = 0 + y = 1.0.$$

答案

$$b = 0.0, \qquad c = 1.0.$$

对应知识点

每次浮点运算后都要舍入，因此改变运算顺序可能改变中间值和最终值。

易错点

不能先按实数代数化简整个表达式；必须按题目给出的括号逐步计算。

4.1 机器指令：题面与思路

题面

objdump 反汇编结果如下：

```
400500: 48 89 d8        mov %rbx,%rax
400503: 48 83 c0 05     add $5,%rax
400507: c3              ret
```

1. 地址 0x400503 处指令占几字节？
2. 三条指令共占几字节？

我的解题思路

反汇编中冒号后、汇编助记符前的每个两位十六进制数就是一个机器代码字节；也可以用相邻指令地址相减交叉检查。

两种做法

1. 直接数每行的机器代码字节；
2. 用下一条指令地址减去当前地址。

两种结果应当一致。

4.1 机器指令：推导与知识点

逐条计数

指令	字节数
mov	3
add	4
ret	1

add = 4 字节，　　总计 = 8 字节.

答案

第二条占 **4** 字节；三条共占 **8** 字节。

对应知识点

x86-64 是变长指令集，每条指令长度可以不同。

易错点

不能默认每条指令都占 4 字节。

0x400507 − 0x400503 = 4 可用于检查。

4.2 寻址方式：题面与思路

题面

已知

$$\texttt{\%rax} = \texttt{0x600},$$
$$\texttt{\%rbx} = \texttt{0x1000},$$
$$\texttt{\%rcx} = 3.$$

执行

```
movq 0x18(%rbx,%rcx,8), %rax
```

求该指令访问的内存地址。

我的解题思路

先套用 $D(R_b, R_i, S)$ 的有效地址公式。注意目标寄存器 %rax 的旧值不参与地址计算。

操作数拆分

$$D = \texttt{0x18}, \quad R_b = \texttt{\%rbx},$$
$$R_i = \texttt{\%rcx}, \quad S = 8.$$

4.2 寻址方式：推导与知识点

代入有效地址公式

$$EA = D + R_b + R_i S.$$

$$\begin{aligned} EA &= \texttt{0x18} + \texttt{0x1000} + 3 \times 8 \\ &= \texttt{0x18} + \texttt{0x1000} + \texttt{0x18} \\ &= \boxed{\texttt{0x1030}}. \end{aligned}$$

答案

访问地址为

$$\texttt{0x1030}.$$

对应知识点

$D(R_b, R_i, S)$ 表示 $D + R_b + R_i S$，比例因子可取 $1, 2, 4, 8$。

易错点

括号外的 0x18 是位移，不能漏加；目标寄存器旧值不参与寻址。

4.2 汇编程序：题面与思路

题面

参数 x 在 %rdi，返回值在 %rax：

```
calc:
    leaq (%rdi,%rdi,2), %rax
    salq $2, %rax
    subq %rdi, %rax
    ret
```

1. leaq 后：A. x；B. $2x$；C. $3x$；D. $4x$。
2. 返回值：A. $7x$；B. $11x$；C. $12x$；D. $13x$。

我的解题思路

不必把它“翻译成完整 C 程序”；给每条指令后的 %rax 写一个关于 x 的代数式即可。

指令含义

- leaq：计算地址表达式；
- salq $2：左移 2 位；
- subq src,dst：$dst \leftarrow dst - src$。

4.2 汇编程序：推导与知识点

符号执行

$$\begin{aligned} \texttt{leaq} \quad &: \quad \%\texttt{rax} = x + 2x = 3x, \\ \texttt{salq \$2} \quad &: \quad \%\texttt{rax} = 3x \cdot 2^2 = 12x, \\ \texttt{subq} \quad &: \quad \%\texttt{rax} = 12x - x = 11x. \end{aligned}$$

(1) **C**, (2) **B**.

答案

第一问选 **C**，第二问选 **B**。

对应知识点

`leaq` 可用于快速完成 $x + kx$ 一类整数运算。

易错点

`leaq` 不访存；AT&T 双操作数顺序是“源，目的”。

4.3 汇编程序：题面与思路

题面

初始时

$$\texttt{\%rdi} = \texttt{0x3000},$$
$$M[\texttt{0x3000}] = 12.$$

```
movq (%rdi), %rax
addq $5, (%rdi)
subq (%rdi), %rax
```

求最终的 %rax 和 $M[\texttt{0x3000}]$（十进制）。

我的解题思路

分开记录“寄存器快照”和“当前内存值”。第二条只修改内存，不会让第一条已经读入 %rax 的 12 自动变化。

推荐草稿格式

每执行一条指令就记一行：

$$(\texttt{\%rax},\ M[\texttt{0x3000}]).$$

这样不会混淆旧的寄存器值与新的内存值。

4.3 汇编程序：推导与知识点

状态跟踪

执行后	%rax	$M[0x3000]$
movq	12	12
addq	12	17
subq	$12 - 17 = -5$	17

答案

$$\%rax = -5, \qquad M[0x3000] = 17.$$

对应知识点

括号表示访存；寄存器保存的是读取当时的值，不与内存自动同步。

易错点

subq src,dst 是 $dst - src$，不能反写成 $src - dst$。

4.4 汇编程序：题面与思路

题面

初始状态：

$$\texttt{\%rdi} = \texttt{0x1000}, \quad \texttt{\%rsi} = 2,$$
$$M[\texttt{0x1000}] = 7, \quad M[\texttt{0x1008}] = 11.$$

```
leaq 8(%rdi), %rcx
movq (%rcx), %rax
leaq (%rax,%rsi,4), %rdx
subq (%rdi), %rdx
movq %rdx, (%rcx)
```

求第二条指令后的 %rax，以及最终 $M[\texttt{0x1008}]$。

我的解题思路

先算地址，再取值；每条指令后只更新它的目的操作数。用 $M[a]$ 明确区分“地址 a”与“地址中的数据”。

跟踪顺序

$$\texttt{\%rcx} \to \texttt{\%rax} \to \texttt{\%rdx} \to M[\texttt{\%rcx}].$$

遇到 leaq 不读内存；遇到括号才解引用。

4.4 汇编程序：推导与知识点

逐条执行

$$\texttt{\%rcx} = \texttt{0x1000} + 8 = \texttt{0x1008},$$
$$\texttt{\%rax} = M[\texttt{0x1008}] = 11,$$
$$\texttt{\%rdx} = 11 + 2 \times 4 = 19,$$
$$\texttt{\%rdx} = 19 - M[\texttt{0x1000}] = 19 - 7 = 12,$$
$$M[\texttt{0x1008}] \leftarrow 12.$$

对应知识点

`leaq` 计算数值；
`movq (%reg),...` 才读取内存。

易错点

最后一条指令会把 12 写回原来存放 11 的位置。

答案

第二条后

$$\texttt{\%rax} = 11;$$

最终

$$M[\texttt{0x1008}] = 12.$$

Ending

谢谢大家！